

DSA Learning Plan and strategy

1 Stick to this Roadmap : [DSA Roadmap](#)

2 If your logic building is weak then first solve these basic questions to master your loopings:

 **STRONG YOUR LOGIC BUILDING .pdf**

3 Master the Basics First –

- Start with fundamental data structures like **string, array, stack, and queue** before moving to advanced topics.
- Focus on **string and array mastery** before solving any question.
- If you are starting from fresh then you have to solve as many questions as you can of strings and arrays.

4 Deep Dive into Each Data Structure:

- First Understand the Data structure concepts.(Follow any playlist on YT)
- Implement all its operations (insert, delete, reverse, etc.) by checking out the solution and understanding the algorithm.
- After this, try to **implement all the operations by yourself**. Write your **own** solution.
- Create a class for each Data structure and create functions for all the operations.
- Operations are the key. If you are able to manipulate operations, you will be able to solve questions effectively later on.

5 Maintain a Detailed Notebook 📓

- Note down everything you learn about that Data structure and different **patterns** you observe in problems.
- Write key operations, variations, and approaches for each topic.
- Even write small code (direct logic) of problems / operations.

6 Find & Solve Pattern-Based Problems

- DO NOT SOLVE RANDOM PROBLEMS.
- Search the particular topic on geeksforgeeks or other sites and find out top 5-10 questions covering different patterns.
- Write them down and **analyze patterns before solving**.
- Understand all different patterns of a data structure. At least solve 5 questions of each pattern.
- After all this try to solve a similar problems by yourself.

7 Create a github Repository

- Create a github Repository.
- Create a folder for each Data Structure and subfolder for operations, easy, medium problems and even for patterns. Also add the problem statement.
- Whatever the code you are solving on leetcode or any platform, create a class in the repo and add the working code with adding a commented question.
- **This should be your own written code.**
- Don't copy paste code here. The best part about this repo is that it will contain all the logic that your mind has formed. Next time your mind will think about the same approach.
- Add comments wherever you feel it is needed.
- Revise concepts from this repo by looking at your own written code. Have it in your mobile and revise whenever you get free time.
- **THIS WILL BE THE BEST WAY TO REVISE.**

HOW TO APPROACH QUESTIONS?

8 Try Before Seeking Help

- Attempt the problem on your own, even if the solution is inefficient.
- Don't think that you are writing ugly solution.
- Forget about time complexity and write the brute force solution even if this is wrong.
- **Do not check solutions immediately.**

Process of solving:

1. **If you can't even write a solution** → Ask ChatGPT/AI tools for an algorithm. Try converting this algorithm into code.
2. **If you wrote something but it's incorrect** → Ask ChatGPT to check where you're wrong. Improve your own solution instead of looking at the perfect one.
3. **Once you get the brute-force approach** → Implement it, then analyze how to improve time complexity.
4. Ask ChatGPT for ways to improve time complexity and create a better solution.
5. Code the better solution.
6. **NEVER EVER LOOK THE PERFECT SOLUTION BEFORE TRYING BY YOURSELF. NEVER.**

9 Optimize Your Code

- Every code in your repo should have 2 functions.
 - Brute force approach function
 - Improved approach function
- After implementation of the Brute force approach, try to improve the time complexity.
- Build an improved solution.

10 Calculate time and space complexity

- For every solution you are writing, try to calculate time and space complexity on your own and note it down with comments.
- By doing this, your skills of calculating time and space complexity will improve.
- Once you solve brute force and calculate time complexity then you can go for creating a better solution with better time complexity.
- For faster time complexity calculation:
 - Follow this guide: [PDF GUIDE _ HOW TO MEASURE TIME COMPLEXITY.pdf](#)
 - Solve all the different patterns including Iterative and Recursive patterns.
 - [PDF Time Complexity Problems Set.pdf](#)
 - Checkout my Youtube Channel for detailed videos.(Coming Soon)
 - [CodeN' on YouTube](#)

11 Short Notes creation:

- Once comfortable and finish the topic, create **short notes summarizing** all patterns for quick revision.
- Use this sample short notes sheet to format your short notes:
 - [How to create own short Notes](#)
- Keep revising your notes.

Directly looking for the solution without approaching the problem is a bad habit that can't even be fixed in 21 days! – NISHCHAL

FOLLOW us on IG - [@codewithnishchal](#)

Subscribe on Youtube - [CodeWithNishchal](#)